

Tối ưu hiệu năng và hệ thống (Performance and Systems Engineering)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (cấu trúc thừa — nguồn của phần lớn chi phí và của phần lớn cơ hội tối ưu); B/09-paradigm-uoc-luong-back-end (lựa chọn kiến trúc quyết định hồ sơ chi phí); C/13-quan-ly-ban-do mục 4A (kiến trúc nhiều luồng — chương này bàn chi tiết kỹ thuật của nó).

TÓM TẮT

Chương này về khoảng cách giữa một thuật toán đúng và một hệ thống chạy được trên phần cứng thật, trong thời gian thật, với ngân sách điện thật. Đó là khoảng cách mà bài báo không mô tả và nó chiếm phần lớn thời gian kỹ thuật của một dự án. Nội dung có bốn phần. Một: thời gian — và luận điểm trung tâm là *phân vị chứ không phải trung bình*, cùng cách xử lý độ trễ và mất khung mà không làm hỏng tính đúng đắn. Hai: kiến trúc đa luồng, nơi ba nhịp độ của C/13 gặp thực tế của khoá, hàng đợi và cạnh tranh dữ liệu — và nơi *tính tất định* nổi lên như một yêu cầu không thể thương lượng. Ba: ngân sách tính toán và bộ nhớ — SIMD, lượng tử hoá, bộ giải thừa, và các quyết định về nơi đặt mỗi phần của pipeline. Bốn: một vòng lặp vật lý mà gần như mọi tài liệu bỏ qua — điện năng, nhiệt, và việc nhiệt làm trôi hiệu chuẩn. Nếu chỉ nhớ một điều: *một hệ nhanh trung bình nhưng thỉnh thoảng chậm là một hệ không dùng được trong vòng điều khiển* — và mọi quyết định trong chương này xoay quanh việc làm cho trường hợp xấu nhất có giới hạn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phân vị (percentile)** — giá trị mà một tỉ lệ nhất định các mẫu nằm dưới nó. Phân vị 99 của thời gian xử lý là ngưỡng mà 99% số khung xử lý xong dưới đó.
- **jitter** — độ biến thiên của thời gian xử lý giữa các khung. Một hệ có jitter lớn khó dùng trong vòng điều khiển ngay cả khi trung bình thấp.
- **tính tất định (determinism)** — cùng đầu vào cho cùng đầu ra, chính xác tới từng bit, qua nhiều lần chạy. Điều kiện để tái hiện một lỗi từ hiện trường.
- **SIMD** — *single instruction multiple data* — chỉ thị xử lý nhiều phần tử cùng lúc. NEON trên ARM, AVX trên x86.
- **allocation-free hot loop** — vòng lặp chính không cấp phát bộ nhớ động. Cấp phát có thời gian không xác định và có thể gây phân mảnh, nên nó là nguồn jitter.
- **thermal throttling** — việc giảm tần số xử lý khi nhiệt độ vượt ngưỡng. Trên thiết bị nhúng không có quạt, đây là ràng buộc thật chứ không phải trường hợp biên.
- **ISP** — *image signal processor* — khối phần cứng xử lý ảnh thô từ cảm biến. Các thao tác của nó (auto-exposure, gamma) ảnh hưởng tới SLAM và thường không điều khiển được đầy đủ.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn xử lý trung bình 18 ms mỗi khung, dư dả cho 30 Hz. Nó vẫn không dùng được trong vòng điều khiển. Nêu đại lượng bạn chưa đo, và giải thích vì sao nó quyết định.
- Q2. Bạn tối ưu và giảm thời gian trung bình từ 18 ms xuống 12 ms. Nêu một trường hợp mà thay đổi đó làm hệ thống *tệ đi*.

- Q3. Đồng nghiệp đề nghị dùng số thực độ chính xác đơn thay vì kép để tăng tốc gấp đôi. Nên điều kiện để làm được điều đó một cách an toàn, và chỉ ra chương nào trong cây cung cấp điều kiện ấy.
- Q4. Một lỗi xảy ra ở hiện trường và bạn có log đầy đủ. Bạn chạy lại trên máy của mình và không tái hiện được. Nên nguyên nhân có khả năng nhất, và nói nó lẽ ra phải được ngăn từ lúc nào.
- Q5. Thiết bị chạy tốt trong mười phút rồi bắt đầu kém chính xác dần, và nó hồi phục sau khi nghỉ. Nên hai cơ chế hoàn toàn khác nhau có thể gây ra điều đó.

1.1 Đặt vấn đề

Ba phần trước nói về thuật toán đúng. Chương này về khoảng cách giữa một thuật toán đúng và một hệ thống chạy được, và khoảng cách ấy lớn hơn nhiều so với vẻ ngoài.

Bài toán gốc: thuật toán chạy trên laptop với bộ dữ liệu đọc từ đĩa không phải thứ chạy trên robot. Ba khác biệt.

Một — thời gian thật. Camera gửi khung ở 30 Hz bất kể bạn đã xử lý xong hay chưa. Không có nút tạm dừng.

Hai — tài nguyên hữu hạn. Một SoC nhúng có vài watt, vài gigabyte RAM, và không có quạt.

Ba — và đây là khác biệt ít được nói nhất: hệ thống là một phần của một hệ thống lớn hơn. Bộ điều khiển cần tư thế đúng lúc; nó không chờ được. Một tư thế chính xác đến sau khi cần thì vô dụng.

Ai gặp bài toán này? Ai đưa SLAM lên thiết bị. Và đây là phần mà kiến thức thuộc về kỹ thuật phần mềm hệ thống chứ không thuộc về thị giác máy tính — một lý do nữa vì sao nó thiếu trong tài liệu SLAM.

Có một cách nhìn xuyên suốt chương, và nó gọn: *mọi thứ trong chương này là về việc làm cho trường hợp xấu nhất có giới hạn.* Không phải làm cho trung bình nhanh hơn — trung bình gần như luôn đủ nhanh. Cái gây ra thất bại là một khung mất một giây, một lần cấp phát gây phân mảnh, một lần throttling làm mọi thứ chậm gấp đôi.

TRỰC GIÁC

Hình dung bạn đi xe đạp và cần nhìn đường.

Nếu bạn nhắm mắt trung bình 0,1 giây mỗi lần chớp mắt, không sao — bạn vẫn đạp được cả ngày.

Nếu thỉnh thoảng bạn nhắm mắt *hai giây*, bạn sẽ đâm vào cột điện. Và điều quan trọng: việc trung bình chỉ có 0,1 giây *không cứu bạn* ở lần hai giây ấy. Các lần nhanh không bù được cho lần chậm.

Đó là toàn bộ lý do phân vị quan trọng hơn trung bình. Với một hệ chỉ cần cho ra kết quả cuối cùng — một phép quét để in 3D chẳng hạn — trung bình là đại lượng đúng, vì tổng thời gian là thứ ta quan tâm. Với một hệ nằm trong vòng điều khiển, mỗi chu kỳ là một hạn chót riêng, và trễ một lần là hỏng một lần.

Có một chuyện thứ hai, cũng về sự bất đối xứng. Nếu bạn phải chọn giữa “nhìn kỹ và biết chính xác mình đang ở đâu, nhưng chậm” và “nhìn lướt và biết đại khái, nhưng kịp”, thì với xe đạp, lựa chọn thứ hai đúng. Một câu trả lời gần đúng đúng lúc hữu ích hơn một câu trả lời chính xác đến muộn. Đây là nguyên tắc chi phối cách thiết kế kiến trúc ở mục 3.

Và chuyện thứ ba, ít ai nghĩ tới. Đạp xe lâu thì bạn nóng người và mệt, và khi mệt thì bạn nhìn kém đi. Cái máy cũng vậy: chạy lâu thì nóng, nóng thì tự giảm tốc, và — điều bất ngờ

— nóng còn làm *ống kính giãn ra*, tức làm sai chính phép đo. Một vòng lặp vật lý mà không tài liệu phần mềm nào nhắc tới.

1.2 Thời gian: phân vị, không phải trung bình

A. Vì sao trung bình không đủ

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất: đại lượng chưa đo là *phân vị 99 của thời gian xử lý*.

Lập luận. Một hệ trong vòng điều khiển có một hạn chót *mỗi chu kỳ*. Nếu chu kỳ là 33 ms và một khung mất 200 ms, thì sáu chu kỳ điều khiển không có dữ liệu mới. Bộ điều khiển phải ngoại suy, và với robot đang di chuyển, sáu chu kỳ ngoại suy có thể là mười centimet sai lệch.

Và điều quan trọng: *các khung nhanh không bù được*. Trung bình là một đại lượng cộng gộp, phù hợp khi ta quan tâm tổng; ở đây mỗi chu kỳ là một sự kiện độc lập với một hạn chót riêng.

Ba nguồn của các khung chậm, theo thứ tự phổ biến trong SLAM.

Một — *keyframe*. Khi một khung được chọn làm keyframe, local mapping chạy, và chi phí nhảy vọt. Nếu nó chạy trên cùng luồng với tracking thì mỗi keyframe là một lần trễ.

Hai — *loop closure*. Hiếm hơn nhưng đắt hơn nhiều: pose graph cộng cập nhật bản đồ có thể mất hàng giây.

Ba — *cấp phát bộ nhớ và thu gom*. Một lần cấp phát lớn có thể mất mili giây và có thời gian không xác định.

B. Vì sao tối ưu trung bình có thể làm tệ đi

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một cái bẫy thật.

Ví dụ cụ thể: bạn thêm một bộ nhớ đệm để tránh tính lại một thứ. Trung bình giảm vì hầu hết khung trúng đệm. Nhưng khi trượt đệm, chi phí cao hơn trước — vì giờ phải tính lại *và* cập nhật đệm. Phân vị 99 tăng lên.

Ví dụ thứ hai, phổ biến hơn: bạn tăng số đặc trưng để chính xác hơn. Trung bình tăng một chút. Nhưng trong các khung khó — nhiều đặc trưng hơn nghĩa là nhiều ứng viên hơn nghĩa là RANSAC lặp lâu hơn — phân vị 99 tăng nhiều.

Quy tắc thực hành: *đo phân vị 99 trước và sau mọi thay đổi hiệu năng*, và coi một thay đổi làm tăng phân vị 99 là một hồi quy kể cả khi trung bình giảm.

C. Xử lý mất khung và độ trễ

Nguyên tắc quan trọng nhất: *không bao giờ được chặn*. Nếu tracking chưa xử lý xong khung này và khung tiếp theo tới, hành vi đúng là *bỏ khung* chứ không phải xếp hàng.

Vì sao xếp hàng sai: hàng đợi làm độ trễ tăng dần và không bao giờ phục hồi. Sau một khung chậm, mọi khung sau đều trễ một chút, và độ trễ tích lũy. Với một vòng điều khiển, một tư thế đúng nhưng cũ 300 ms tệ hơn một tư thế kém chính xác hơn nhưng mới.

Ba chi tiết cài đặt đáng nhớ. *Một*: hàng đợi giữa các luồng nên có kích thước *một*, và ghi đè thay vì xếp hàng. *Hai*: mỗi khung phải mang *mốc thời gian thu*, không phải thời gian xử lý — mọi nhân tử trong đồ thị dùng mốc thu. *Ba*: đầu ra phải kèm *tuổi* của nó, để module phía trên biết mà chiết khấu; đây là phần mở rộng của nguyên tắc ở D/19 mục 6C.

1.3 Kiến trúc đa luồng và tính tất định

A. Ba luồng, ba nhịp độ

C/13 mục 4A nêu ba nhịp độ: tracking ở tần số khung, local mapping ở tần số keyframe, loop closing hiếm và lâu. Chương này bàn chi tiết kỹ thuật.

Nguyên tắc phân chia: *luồng có hạn chót cứng phải được cô lập*. Tracking có hạn chót; hai luồng kia không. Nên tracking không bao giờ được chờ hai luồng kia, kể cả khi nó cần dữ liệu mà chúng đang sửa.

Ba mẫu thiết kế giải quyết điều đó.

Bản sao đọc. Tracking làm việc trên một bản sao của bản đồ cục bộ, được cập nhật ở các điểm an toàn. Tồn bộ nhớ, nhưng tracking không bao giờ bị chặn.

Hoán đổi con trỏ. Local mapping xây dựng phiên bản mới của một cấu trúc rồi hoán đổi con trỏ trong một thao tác nguyên tử. Tracking thấy hoặc phiên bản cũ hoặc mới, không bao giờ thấy trạng thái nửa vời.

Hàng đợi một chiều. Tracking gửi keyframe cho local mapping qua một hàng đợi và *không chờ kết quả*. Kết quả quay lại qua một trong hai cơ chế trên.

B. Tính tất định là một yêu cầu, không phải xa xỉ

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và là luận điểm mà tôi cho là quan trọng nhất của chương.

Kịch bản: một robot ở hiện trường gặp lỗi. Bạn có log đầy đủ — mọi ảnh, mọi phép đo IMU, mọi mốc thời gian. Bạn chạy lại và *không tái hiện được*.

Nguyên nhân có khả năng nhất: *hệ thống không tất định*. Thứ tự luồng khác nhau làm keyframe khác được chọn; RANSAC với seed khác cho inlier khác; một khung bị bỏ ở lần chạy này mà không bị bỏ ở lần kia.

Hậu quả: bạn không debug được. Bạn có thể đoán, sửa, và không bao giờ biết mình đã sửa đúng chưa. Với một lỗi xảy ra một lần trong nghìn giờ, đây là sự khác biệt giữa sửa được và không.

Ba việc phải làm, và từ đầu dự án chứ không phải khi gặp lỗi.

Một — seed cố định và có thể tái lập. Mọi nguồn ngẫu nhiên (RANSAC, khởi tạo) phải lấy seed từ một nguồn ghi vào log.

Hai — chế độ chạy lại tất định. Một chế độ trong đó dữ liệu được đưa vào theo mốc thời gian ghi trong log, không theo thời gian thực, và các luồng chạy theo một thứ tự cố định. Chế độ này chạy chậm hơn thời gian thực và điều đó chấp nhận được — mục đích của nó là chẩn đoán.

Ba — ghi lại mọi quyết định không tất định. Khung nào bị bỏ, keyframe nào được chọn, cổng nào đóng mở. Nếu không tái lập được hoàn toàn, ít nhất có thể tái lập các quyết định.

Cái giá của tính tất định là thật: nó hạn chế một số tối ưu (xử lý song song không thứ tự), và nó có thể chậm hơn vài phần trăm. Đó là một cái giá đáng trả, và nó là một trong những lý do chính đáng nhất để chấp nhận chạy chậm hơn một chút — cùng lập luận với việc chọn ORB thay vì một front-end học được ở B/07 mục 2B.

1.4 Ngân sách tính toán và bộ nhớ

A. Biết chi phí nằm ở đâu

Trước khi tối ưu, đo. Hồ sơ chi phí điển hình của một hệ SLAM thị giác, theo thứ tự thường gặp: trích và mô tả đặc trưng thường là mục lớn nhất; khớp và RANSAC; bundle adjustment cục bộ; và các phần còn lại.

Tôi *không* đưa ra tỉ lệ phần trăm cụ thể, vì chúng phụ thuộc hoàn toàn vào độ phân giải, số đặc trưng, kích thước cửa sổ và phần cứng. Điều đáng nói là một nguyên tắc: *đo trước khi tối ưu*, và đo trên *phần cứng mục tiêu* — hồ sơ chi phí trên laptop x86 và trên SoC ARM khác nhau đáng kể, vì tỉ lệ giữa tốc độ CPU và băng thông bộ nhớ khác nhau.

B. Các kỹ thuật, theo thứ tự tỉ lệ lợi ích trên công sức

Giảm công việc trước khi tăng tốc công việc. Cách rẻ nhất để làm một phép tính nhanh hơn là không làm nó. Ví dụ cụ thể trong SLAM: thu hẹp vùng tìm kiếm bằng tiên nghiệm chuyển động (B/07 mục 5C) giảm chi phí khớp hàng chục lần; chọn keyframe tiết kiệm (C/13 mục 2) giảm chi phí BA theo lũy thừa ba.

SIMD cho các vòng lặp nóng. Tính descriptor, tính khoảng cách Hamming, warp ảnh — các thao tác này song song hoá hoàn hảo ở mức dữ liệu. Lợi ích lớn và công sức vừa phải.

Vòng lặp nóng không cấp phát. Cấp phát trước mọi bộ đệm và tái sử dụng. Lợi ích chính không phải tốc độ trung bình mà là *giảm jitter* — tức đúng thứ mục 2 nói là quan trọng.

Bộ giải thừa tối ưu. Khai thác cấu trúc khối cụ thể của bài toán thay vì dùng bộ giải thừa tổng quát (A/04 mục 4). Công sức lớn, lợi ích lớn.

Độ chính xác đơn. Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Nó nhanh gấp đôi và tốn nửa bộ nhớ, nhưng nó nguy hiểm với bài toán có số điều kiện lớn — và BA có số điều kiện lớn (B/09 mục 5A). Điều kiện để làm an toàn: *dùng dạng căn bậc hai*, vốn có số điều kiện là căn bậc hai của dạng hệ chuẩn [1]. B/09 mục 5A là chương cung cấp điều kiện ấy, và đây là một ví dụ đẹp về việc một lựa chọn toán học mở ra một tối ưu hệ thống.

C. Đặt cái gì ở đâu

Trên SoC hiện đại có nhiều khối tính toán, và việc phân bổ là một quyết định thiết kế.

CPU cho logic điều khiển và bộ giải thừa — các phép toán có nhiều nhánh và truy cập bộ nhớ không đều, nơi GPU không giúp. GPU cho các thao tác dày và đều — trích đặc trưng, mạng nơ-ron, stereo dày. NPU cho mạng, khi có. ISP cho tiền xử lý — nhưng với một cảnh báo quan trọng.

Cảnh báo về ISP: nó thường thực hiện các thao tác mà SLAM không muốn — auto-exposure thay đổi giữa các khung (vấn đề cho direct, A/02 mục 5B), khử nhiễu làm mất chi tiết nhỏ (vấn đề cho detector), và gamma phi tuyến (vấn đề cho mô hình quang trắc). Và các thao tác ấy thường không tắt được hoặc không điều khiển được đầy đủ.

Quy tắc: *lấy ảnh càng gần raw càng tốt*, và nếu phải dùng ISP thì *ghi lại tham số của nó cho mỗi khung* — thời gian phơi sáng và gain phải là một phần của dữ liệu, vì A/02 cần chúng. Đây là một yêu cầu với nhà cung cấp module camera, và nó nên được nêu *trước khi chọn module*.

1.5 Điện năng, nhiệt, và một vòng lặp bị bỏ quên

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và là mục mà tôi cho là đóng góp riêng của chương.

Hai cơ chế hoàn toàn khác nhau giải thích hiện tượng “chạy tốt mười phút rồi kém dần, hồi phục sau khi nghỉ”.

Cơ chế một — thermal throttling. SoC nóng lên, tần số giảm, thời gian xử lý tăng. Hệ quả theo mục 2: nhiều khung bị bỏ hơn, cửa sổ BA phải thu nhỏ, độ trễ tăng. Đây là cơ chế mà mọi kỹ sư nhúng đều biết.

Cơ chế hai — trôi hiệu chuẩn do nhiệt. Ống kính và khung giãn nở, tiêu cự đổi vài phần vạn, và với stereo thì đường cơ sở cùng góc giữa hai camera đổi. Theo A/02 mục 4C, sai số hiệu chuẩn chảy vào tư thế và cấu trúc như một sai lệch có hệ thống.

Hai cơ chế cho cùng triệu chứng và cần hai cách chữa hoàn toàn khác nhau. Cơ chế một chữa bằng quản lý nhiệt và giảm tải; cơ chế hai chữa bằng hiệu chuẩn trực tuyến (C/14 mục 3) hoặc bằng mô hình bù theo nhiệt độ.

Phép thử phân biệt, và nó rẻ: *ghi nhiệt độ vỏ máy cùng với các đại lượng giám sát.* Nếu thời gian xử lý tăng theo nhiệt thì là cơ chế một; nếu *phần dư có cấu trúc* hoặc *tham số hiệu chuẩn trực tuyến* tương quan với nhiệt thì là cơ chế hai. Hai dấu hiệu ấy độc lập, nên phép thử phân biệt được rõ ràng — và cả hai có thể xảy ra cùng lúc.

Điều đáng nói: vòng lặp “điện năng → nhiệt → trôi ống kính → trôi hiệu chuẩn → sai số thị giác” gần như không xuất hiện trong tài liệu SLAM, vì nó nằm ở giao của ba lĩnh vực — nhiệt học, quang cơ, và thị giác — và không ai sở hữu nó. Tôi nêu nó ở đây vì tôi cho rằng nó là một trong những nguyên nhân hay bị chẩn đoán nhầm nhất trong triển khai thật (*đây là suy luận của tôi từ A/02 mục 4C và C/14 mục 4B; tôi không có số liệu đo và không dẫn được nguồn nào đã đo vòng lặp này một cách hệ thống*).

Hệ quả thiết kế, và nó thuộc về D/21: *đặt một cảm biến nhiệt gần module camera và ghi giá trị của nó cùng mọi dữ liệu.* Chi phí vài xu; giá trị chẩn đoán rất cao; và không thêm được sau khi đã mở khuôn.

CẦN NHỚ

Bốn quy tắc của chương này, xếp theo mức quan trọng.

Một — đo phân vị, không đo trung bình. Một hệ nhanh trung bình nhưng thỉnh thoảng chậm là hệ không dùng được trong vòng điều khiển, vì các lần nhanh không bù được cho lần chậm. Coi một thay đổi làm tăng phân vị 99 là hồi quy kể cả khi trung bình giảm.

Hai — không bao giờ chặn; bỏ khung thay vì xếp hàng. Hàng đợi làm độ trễ tăng dần và không phục hồi. Một tư thế kém chính xác đúng lúc hữu ích hơn một tư thế chính xác đến muộn.

Ba — tắt định từ đầu dự án. Không có nó thì không tái hiện được lỗi hiện trường, và với một lỗi xảy ra một lần trong nghìn giờ, đó là khác biệt giữa sửa được và không. Cái giá vài phần trăm tốc độ là đáng trả.

Bốn — ghi nhiệt độ. Vài xu cho một cảm biến, và nó là thứ phân biệt được hai cơ chế suy giảm hoàn toàn khác nhau mà nếu không có thì bạn sẽ chẩn đoán nhầm.

Và một quy tắc bao trùm: **mọi thứ trong chương này là về giới hạn trường hợp xấu nhất, không về cải thiện trung bình.** Trung bình gần như luôn đủ nhanh.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Basalt [2]	Dạng căn bậc hai; chất lượng kỹ thuật phần mềm rất cao	Mục 4B — ví dụ về việc lựa chọn toán học mở ra tối ưu hệ thống

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [3]	Ba luồng với hàng đợi và khoá	Mục 3A; đọc mã để thấy các mẫu thiết kế thật, gồm cả chỗ chúng phức tạp hơn mô tả trong bài báo
VINS-Mono [4]	Tách luồng estimator và luồng loop closure	Kiến trúc gọn, dễ đọc để học nguyên tắc cô lập luồng có hạn chót
Ceres [5]	Bộ giải thưa với nhiều lựa chọn ordering và linear solver	Mục 4B; thử các lựa chọn trên bài toán của bạn thay vì dùng mặc định
OpenCV	Nhiều hàm đã có phiên bản SIMD	Mục 4B; kiểm xem build của bạn có bật đúng cờ kiến trúc không — một lỗi phổ biến

MINI PROJECT SLAM · Đo hồ sơ thời gian trên phần cứng mục tiêu, rồi tấn công phân vị 99 · 4 ngày

Mục tiêu	chuyển từ “hệ của tôi chạy 30 Hz” sang “phân vị 99 của tôi là X ms, và nguồn của nó là Y ”.
Dữ liệu	một chuỗi dài (ít nhất mười phút) chạy trên <i>phần cứng mục tiêu</i> , không phải laptop. Nếu chưa có phần cứng mục tiêu, dùng thứ gần nhất và ghi rõ rằng kết quả sẽ khác.
Các bước	đo thời gian của từng giai đoạn pipeline cho <i>mỗi khung</i> và lưu lại toàn bộ; vẽ histogram và phân vị 50, 95, 99, 99,9; xác định các khung ở đuôi và <i>tìm ra chúng có gì chung</i> — có phải keyframe không, có loop closure không, có phải lần cấp phát lớn không; ghi nhiệt độ trong suốt và vẽ thời gian xử lý theo nhiệt độ; rồi chọn <i>một</i> nguồn ở đuôi và sửa nó, đo lại.
Xong khi	bạn có một biểu đồ phân vị trước và sau, và một lời giải thích cho <i>từng</i> đỉnh ở đuôi. Điều quan trọng nhất: bạn phải trả lời được câu “khung chậm nhất của tôi chậm vì cái gì” cho ít nhất ba nguồn khác nhau. Nếu đường cong thời gian theo nhiệt độ có xu hướng tăng, bạn vừa đo được cơ chế một ở mục 5 trên chính thiết bị của mình.
Nối với	D/21-tu-prototype-len-mass-production biến các đại lượng này thành giám sát fleet; D/19-ben-vung-trong-the-gioi-thuc mục 6 dùng chúng làm đầu vào cho phát hiện thất bại.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đại lượng chưa đo là *phân vị 99 của thời gian xử lý*. Một hệ trong vòng điều khiển có hạn chót *mỗi chu kỳ*, và các khung nhanh không bù được cho khung chậm — trung bình là đại lượng cộng gộp, phù hợp khi ta quan tâm tổng, nhưng ở đây mỗi chu kỳ là một sự kiện độc lập với hạn chót riêng. Nếu một khung mất 200 ms thì sáu chu kỳ điều khiển không có dữ liệu mới, và với robot đang di chuyển đó có thể là mười centimet sai lệch. Ba nguồn thường gặp của khung chậm: keyframe kích hoạt local mapping, loop closure, và cấp phát bộ nhớ.
- A2. Ví dụ: bạn thêm một bộ nhớ đệm. Trung bình giảm vì hầu hết khung trúng đệm, nhưng khi *trượt* đệm thì chi phí cao hơn trước — phải tính lại *và* cấp nhật đệm — nên phân vị 99 tăng. Ví dụ thứ hai phổ biến hơn: tăng số đặc trưng để chính xác hơn làm trung bình tăng một chút, nhưng trong các khung khó thì nhiều ứng viên hơn nghĩa là RANSAC lặp lâu hơn, và phân vị 99 tăng nhiều. Quy tắc: đo phân vị 99 trước và sau mọi thay đổi hiệu năng, và coi việc nó tăng là một hồi quy kể cả khi trung bình giảm.
- A3. Điều kiện: *dùng dạng căn bậc hai* thay vì hệ chuẩn. Vì $\text{cond}(\Lambda) = \text{cond}(R)^2$, việc dựng Λ tương minh phá huỷ một nửa số chữ số có nghĩa — với bài toán BA có số điều kiện 10^8 , độ chính xác đơn (khoảng bảy chữ số) không đủ. Làm việc với R giữ lại số điều kiện ở mức căn bậc hai, và [1] cho thấy điều đó cho phép dùng độ chính xác đơn trong nhiều trường hợp. Chương cung cấp điều kiện là B/09 mục 5A — một ví dụ đẹp về việc một lựa chọn toán học mở ra một tối ưu hệ thống.

- A4. Nguyên nhân có khả năng nhất: *hệ thống không tắt định* — thứ tự luồng khác làm keyframe khác được chọn, RANSAC với seed khác cho inlier khác, một khung bị bỏ ở lần này mà không bị ở lần kia. Hậu quả: bạn không debug được; bạn đoán, sửa, và không bao giờ biết đã sửa đúng chưa. Lẽ ra phải ngăn từ đầu dự án, bằng ba việc: seed cố định ghi vào log; một chế độ chạy lại tắt định trong đó dữ liệu vào theo mốc thời gian ghi trong log và luồng chạy theo thứ tự cố định; và ghi lại mọi quyết định không tắt định (khung nào bị bỏ, keyframe nào được chọn). Cái giá vài phần trăm tốc độ là đáng trả.
- A5. Hai cơ chế. (1) *Thermal throttling*: SoC nóng, tần số giảm, thời gian xử lý tăng, nhiều khung bị bỏ hơn. (2) *Trôi hiệu chuẩn do nhiệt*: ống kính và khung giãn nở, tiêu cự đổi, và với stereo thì đường cơ sở cùng góc giữa hai camera đổi — sai số hiệu chuẩn chảy vào tư thế như một sai lệch có hệ thống (A/02 mục 4C). Hai cơ chế cho cùng triệu chứng và cần hai cách chữa ngược nhau: quản lý nhiệt so với hiệu chuẩn trực tuyến. Phép thử phân biệt rõ: ghi nhiệt độ vỏ máy; nếu *thời gian xử lý tăng theo nhiệt* thì là (1), nếu *phần dư có cấu trúc* hoặc tham số hiệu chuẩn trực tuyến tương quan với nhiệt thì là (2). Cả hai có thể xảy ra cùng lúc.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về việc đạp xe và cần nhìn đường.

Nếu trung bình bạn chớp mắt một phần mười giây, không sao cả. Nhưng nếu thỉnh thoảng bạn nhắm mắt hai giây, bạn sẽ đâm vào cột điện — và việc trung bình chỉ có một phần mười giây *không cứu bạn* ở lần hai giây ấy.

Đó là chuyện quan trọng nhất của chương. Khi một cái máy phải trả lời liên tục cho một thứ khác đang chuyển động, thì thứ đáng lo không phải là nó trả lời nhanh bao nhiêu *trung bình*, mà là lần chậm nhất của nó chậm bao lâu. Các lần nhanh không bù được cho lần chậm.

Chuyện thứ hai cũng về sự lựa chọn giữa nhanh và đúng. Nếu phải chọn giữa “nhìn thật kỹ rồi mới trả lời” và “nhìn lướt nhưng trả lời kịp”, thì với người đang đạp xe, lựa chọn thứ hai đúng. Một câu trả lời gần đúng đúng lúc hữu ích hơn một câu trả lời chính xác đến muộn. Và có một hệ quả cụ thể: khi máy xử lý không kịp, đừng để nó xếp hàng — hãy để nó *bỏ qua* và nhìn tấm ảnh mới nhất. Xếp hàng thì nó sẽ luôn trả lời về chuyện đã xảy ra vài giây trước.

Chuyện thứ ba ít ai nghĩ tới nhưng lại là chuyện tốn nhiều thời gian nhất. Giả sử máy hỏng ở nhà khách hàng và bạn có đầy đủ ghi chép về mọi thứ nó thấy. Bạn chạy lại y hệt trên máy của mình — và nó chạy bình thường. Chuyện này xảy ra khi máy *không chạy giống nhau qua hai lần*, vì có chỗ nào đó dựa vào may rủi hoặc vào thứ tự các việc chạy song song. Lúc đó bạn không thể sửa, vì bạn không tái hiện được lỗi. Và cách duy nhất tránh là làm cho nó chạy giống hệt nhau *ngay từ đầu* — thêm vào sau thì gần như không làm được.

Và chuyện cuối, thuộc về vật lý chứ không phải phần mềm. Chạy lâu thì máy nóng. Nóng thì nó tự chạy chậm lại — điều ai cũng biết. Nhưng nóng còn làm *ống kính giãn ra một chút*, tức làm sai chính phép đo — điều gần như không ai nhắc. Hai chuyện này gây ra cùng một triệu chứng: máy chạy tốt mười phút rồi kém dần, rồi hồi phục sau khi nghỉ. Chúng cần hai cách chữa hoàn toàn khác nhau, và cách phân biệt chỉ tốn một cái cảm biến nhiệt giá vài xu.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *phải quyết định các chuyện này từ rất sớm*. Tính tắt định không thêm vào được sau; cảm biến nhiệt không gắn thêm được sau khi đã mở khuôn nhựa. Đó là những quyết định phải đưa ra lúc chưa ai thấy chúng cần thiết, và đó là lý do chúng hay bị bỏ qua.

Một câu: *Đừng hỏi trung bình bao nhanh — hỏi lần chậm nhất chậm bao lâu.*

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *phân vị quan trọng hơn trung bình* — đúng cho hệ nằm trong vòng điều khiển và *sai* cho hệ xử lý ngoại tuyến, nơi tổng thời gian là thứ đáng quan tâm. Nó cũng yếu đi nếu module phía trên có khả năng ngoại suy tốt và chịu được độ trễ — một bộ điều khiển có mô hình

động học tốt có thể chịu được vài chu kỳ thiếu dữ liệu. Nên phát biểu chặt hơn: *phân vị quan trọng theo tỉ lệ với mức mà module phía trên không chịu được độ trễ*.

Mệnh đề thứ hai — *tính tất định là yêu cầu bắt buộc* — bị bác bỏ nếu trong thực tế, phần lớn lỗi hiện trường tái hiện được mà không cần nó, hoặc nếu các kỹ thuật ghi log đủ chi tiết thay thế được. Tôi cho là không, nhưng mức độ phụ thuộc vào tần suất các lỗi hiếm — và với một hệ ổn định thì có thể chúng đủ hiếm để không đáng đầu tư. Đây là một đánh đổi phụ thuộc dự án.

Mệnh đề thứ ba — *vòng lặp nhiệt gây trôi hiệu chuẩn* — là suy luận của tôi từ hai chương khác, và tôi đã ghi rõ rằng *không có số liệu đo và không dẫn được nguồn nào đã đo nó một cách hệ thống*. Đây là mệnh đề có rủi ro sai cao nhất trong chương. Nó dễ kiểm: ghi nhiệt độ và tham số hiệu chuẩn trực tuyến trên một thiết bị thật trong vài giờ, và xem chúng có tương quan không. Nếu không, mệnh đề sai và tôi nên rút nó. Phép đo này nằm trong 99-chua-biet.md như một mục ưu tiên cao, vì nó rẻ và vì kết quả đổi cách chẩn đoán.

1.7 Kết nối

Nối lên trên. A/04-uoc-luong-map-va-bundle-adjustment là nguồn của cả chi phí lẫn cơ hội tối ưu lớn nhất — cấu trúc thừa. B/09-paradigm-uoc-luong-back-end mục 5A cung cấp điều kiện cho việc dùng độ chính xác đơn, và nói chung là chỗ lựa chọn kiến trúc quyết định hồ sơ chi phí. C/13-quan-ly-ban-do mục 4A nêu kiến trúc ba luồng mà chương này bàn chi tiết kỹ thuật.

Nối ngang. D/19-ben-vung-trong-the-gioi-thuc mục 6 đề nghị bốn tầng giám sát; chương này nhắc rằng chúng không miễn phí và phải nằm trong ngân sách. Ngược lại, mục 5 ở đây cho D/19 một chế độ hồng thuộc nhóm bốn mà chương đó không nêu: trôi do nhiệt theo chu kỳ hoạt động chứ không theo tuổi thiết bị.

Nối xuống dưới. D/21-tu-prototype-len-mass-production là chỗ các quyết định ở đây trở thành yêu cầu phân cứng: chọn SoC theo ngân sách nhiệt chứ không chỉ theo hiệu năng đỉnh, yêu cầu truy cập ảnh gần raw từ nhà cung cấp module, và đặt cảm biến nhiệt gần camera. Cả ba phải quyết *trước khi chốt thiết kế*, và đó là lý do chương 21 khuyến nghị đọc chương này sớm hơn thứ tự tự nhiên của nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao trung bình không đủ cho một hệ trong vòng điều khiển, và nêu ba nguồn thường gặp của các khung ở đuôi.
2. Nêu một thay đổi làm giảm thời gian trung bình mà tăng phân vị 99, và giải thích cơ chế.
3. Vì sao bỏ khung đúng hơn xếp hàng? Nêu ba chi tiết cài đặt đi kèm nguyên tắc đó.
4. Nêu ba việc phải làm để có tính tất định, và nói vì sao chúng phải làm từ đầu dự án.
5. Điều kiện để dùng độ chính xác đơn an toàn là gì, và chương nào cung cấp điều kiện ấy?
6. Nêu cảnh báo về ISP và hai yêu cầu nên đặt ra với nhà cung cấp module camera.
7. Hai cơ chế gây “chạy tốt rồi kém dần rồi hồi phục” là gì, và phép thử nào phân biệt chúng?

Nguồn

- [1] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [2] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [5] S. Agarwal, K. Mierle, and The Ceres Solver Team, *Ceres Solver*, phần mềm mã nguồn mở, mức tin B; tài liệu phần mềm, không phải công trình phản biện, 2022.